

RELATÓRIO TÉCNICO: SISTEMA DE RESERVA DE SALAS (UPE)

Disciplina: Banco de Dados

Professor: Dr. Carlos Melo

Curso: Engenharia de Software – Campus Garanhuns

Grupo: Gabriel Acácio Vasconcelos Sampaio

1. O Problema e a Justificativa do Projeto

O cotidiano acadêmico no Campus Garanhuns da Universidade de Pernambuco (UPE) envolve uma dinâmica intensa de uso de espaços: salas de aula, laboratórios de informática e auditórios são disputados diariamente por professores, alunos e servidores. Atualmente, a falta de um sistema centralizado pode gerar problemas, como o choque de horários para o mesmo espaço e a dificuldade de reportar problemas estruturais (ar-condicionado quebrado, projetor com defeito, etc.).

Este projeto resolve esse problema ao mapear digitalmente a estrutura física de blocos e salas da instituição, permitindo um agendamento seguro de reservas e o rastreamento de ocorrências de manutenção em tempo real.

Decisão de Modelagem (Simplificação de Escopo)

Durante a fase de análise, optei por **remover a entidade "Equipamento"** que havia cogitado inicialmente. Tomei essa decisão técnica para garantir a atomicidade e a integridade do fluxo crítico do sistema (que é o agendamento do espaço físico). Em vez de criar tabelas redundantes para gerenciar cabos ou controles remotos, centralizei os problemas estruturais na tabela OCORRENCIA, mantendo o banco de dados enxuto, normalizado e focado na real necessidade da comunidade acadêmica.

2. Atendimento aos Requisitos Técnicos (Meta de Elementos)

Para cumprir a meta estipulada pelo professor de desenvolver um banco de dados robusto com **no mínimo 20 elementos**, a estrutura física foi expandida além das tabelas relacionais padrão, englobando lógica de negócios automatizada diretamente no servidor PostgreSQL.

A contagem exata dos meus 20 elementos consiste em:

- **7 Tabelas Relacionais (DDL):** Criadas respeitando as restrições de integridade, chaves primárias compostas e estrangeiras (BLOCO, SIA, USUARIO, PROFESSOR, ALUNO, RESERVA, OCORRENCIA).
- **5 Views (Consultas Customizadas):** Desenvolvidas para simplificar relatórios complexos que exigem múltiplos JOINS, isolando a complexidade para o usuário final (ex: v_relatorio_reservas, v_perfil_alunos).
- **4 Elementos de Trigger (Automação):** Sendo 2 funções de gatilho e 2 Triggers ativos responsáveis por impedir erros humanos críticos, como o choque de horários na mesma sala e a inserção de horários retroativos.
- **4 Stored Procedures (Procedimentos Armazenados):** Rotinas criadas para otimizar operações repetitivas no banco, como o cancelamento automático de reservas (sp_cancelar_reserva) e inserções rápidas.

3. Teste de Integração com a Camada de Aplicação

Como o ambiente local de desenvolvimento conta com o ambiente experimental Python 3.15, eu optei por realizar a validação da camada de aplicação utilizando o driver nativo sqlite3 para simular as regras de negócio lógicas e o mapeamento das tabelas em memória RAM.

O teste disparou a criação da estrutura de dados e inseriu registros simulados com sucesso, retornando o código de encerramento padrão do sistema (exit code 0), o que valida a viabilidade de integração do banco de dados com sistemas externos de mercado.

```
[SUCESSO] Conexão com a camada de aplicação estabelecida!
Dados retornados do teste: ID: 1 | Nome: Bloco A - Garanhuns

--- TESTE CONCLUÍDO COM ÊXITO ---

Process finished with exit code 0
```

4. Estrutura do Repositório (GitHub)

O projeto foi comitado de forma limpa na branch principal (**main**) do repositório, contendo a seguinte organização de arquivos:

- README.md: Capa de apresentação e instruções de execução.
- modelo_conceitual.png: Diagrama conceitual de entidades e relacionamentos.
- modelo_logico.png: Diagrama lógico com a tipagem e chaves estrangeiras.
- script_banco_completo.sql: Arquivo unificado com todo o código SQL executável no pgAdmin.
- teste_app.py: Código em Python que valida a camada de aplicação.
- documentacao.pdf: Este relatório técnico descritivo.